Predicting Extended Length of Stay in Intensive Care Units Using MIMIC-IV

Unison Park & Brianna Gamble

Abstract

Prolonged intensive care unit (ICU) stays create significant clinical, financial, and operational challenges, necessitating early risk prediction for efficient resource allocation and improved patient outcomes. This project utilized the MIMIC-IV database to develop predictive models for extended ICU length of stay (≥7 days) using only the first 12 hours of admission data. Three supervised machine learning algorithms: logistic regression (LR), XGBoost (XGB), and support vector machine (SVM), were trained and optimized using cross-validation and hyperparameter tuning, targeting the AUPRC metric to address class imbalance. The XGB model achieved the highest discrimination (AUPRC = 0.4329). Explainable AI (XAI) identified respiratory and metabolic instability as the most influential predictors. However, fairness evaluation revealed significant performance disparities across ethnicity and age subgroups, with high underdiagnosis rates for several racial/ethnic minorities. The findings demonstrate that accurate, interpretable predictions can be generated from early physiologic data, potentially supporting proactive care management.

I. Introduction

Length of stay (LOS) is a key indicator of ICU performance and patient prognosis. Patients who remain in the ICU for more than 7 days often experience higher complication rates, increased healthcare costs, and greater mortality risk (Johnson et al., 2023). From an operational standpoint, prolonged ICU occupancy limits bed availability and strains staffing resources, creating challenges in high-volume tertiary hospitals.

Predicting extended LOS within the first few hours of admission could enable targeted interventions such as early mobilization, discharge planning, and allocation of specialist resources. The MIMIC-IV database (Johnson et al., 2023) provides a rich repository of de-identified ICU data that captures vital signs, laboratory values, and treatment variables recorded in near real time. This project aimed to apply supervised machine-learning techniques to predict prolonged ICU stays and to improve

performance. Explainable artificial intelligence (XAI), fairness, and distribution-shift methods were tested and evaluated to clarify how model features influence outcomes. The objectives were to 1) build models predicting extended LOS (≥ 7 days) using only the first 12 hours of ICU data, 2) compare performance between linear and tree-based models and improve them, 3) conduct XAI, fairness, and distribution-shift analyses on the highest performing model.

II. Results

Demographic Analysis

The demographic statistics included in our study are age, gender, ethnicity, and insurance are shown in Table S1. Among the patients, the age was virtually identical across the sets, averaging around 63.0 years. Males slightly outnumber females, constituting 57.3% of the training set and similar proportions in the other sets. Racially, the cohort was predominantly White, Insurance coverage was dominated by Other and Medicare. Among the comorbidities, essential hypertension was the most common (around 43%).

Model Performance

Metric	LR (Old)	LR (New)	SVM (Old)	SVM (New)	XGB (Old)	XGB (New)
BCE Loss	1.17	0.5072	1.21	0.5691	1.33	0.4509
AU-ROC	0.77	0.8316	0.77	0.8313	0.78	0.8496
AU-PRC	0.31	0.3924	0.31	0.3919	0.31	0.4329
Accuracy	0.77	0.7719	0.77	0.776	0.72	0.7989
Precision	0.27	0.2934	0.28	0.2979	0.24	0.3215
Recall	0.63	0.7378	0.63	0.739	0.7	0.7189
Specificity	0.79	0.7762	0.79	0.7807	0.72	0.8089
NPV	0.94	0.9592	0.94	0.9596	0.95	0.9581

ECE	0.25	0.2583	0.32	0.3272	0.14	0.2307
MCE	0.5	0.4592	0.47	0.3751	0.25	0.4193

Table 1 . Comparative Analysis of Model Performance: Project 2 (old) vs. Final Project (new)

Table 1 presents a comprehensive comparison of model performance for three distinct classification algorithms across both the Project 2 and Final Project training methodologies, likely representing baseline and optimized versions, respectively. The "New" models consistently demonstrate superior performance across metrics, highlighted by a substantial reduction in the Binary Cross-Entropy (BCE) Loss (1.33 to 0.4509 for XGB) and increases in the AUROC curve, with XGB achieving the highest score of 0.8496. Furthermore, the improvements are also evident in the AUPRC (0.4329) and the Recall metric (0.7189), which is crucial for identifying positive cases, while maintaining high Negative Predictive Value (NPV); for instance, the new XGB model yields the best overall Accuracy at 0.7989, with a slight increase in Expected Calibration Error (ECE) compared to its counterpart, indicating potential trade-offs between discrimination and calibration in the optimized models.

XAI

Figure 2. SHAP values for a model's prediction of 0.661, showing which features pushed the prediction up (orange) or down (blue) from the base value.

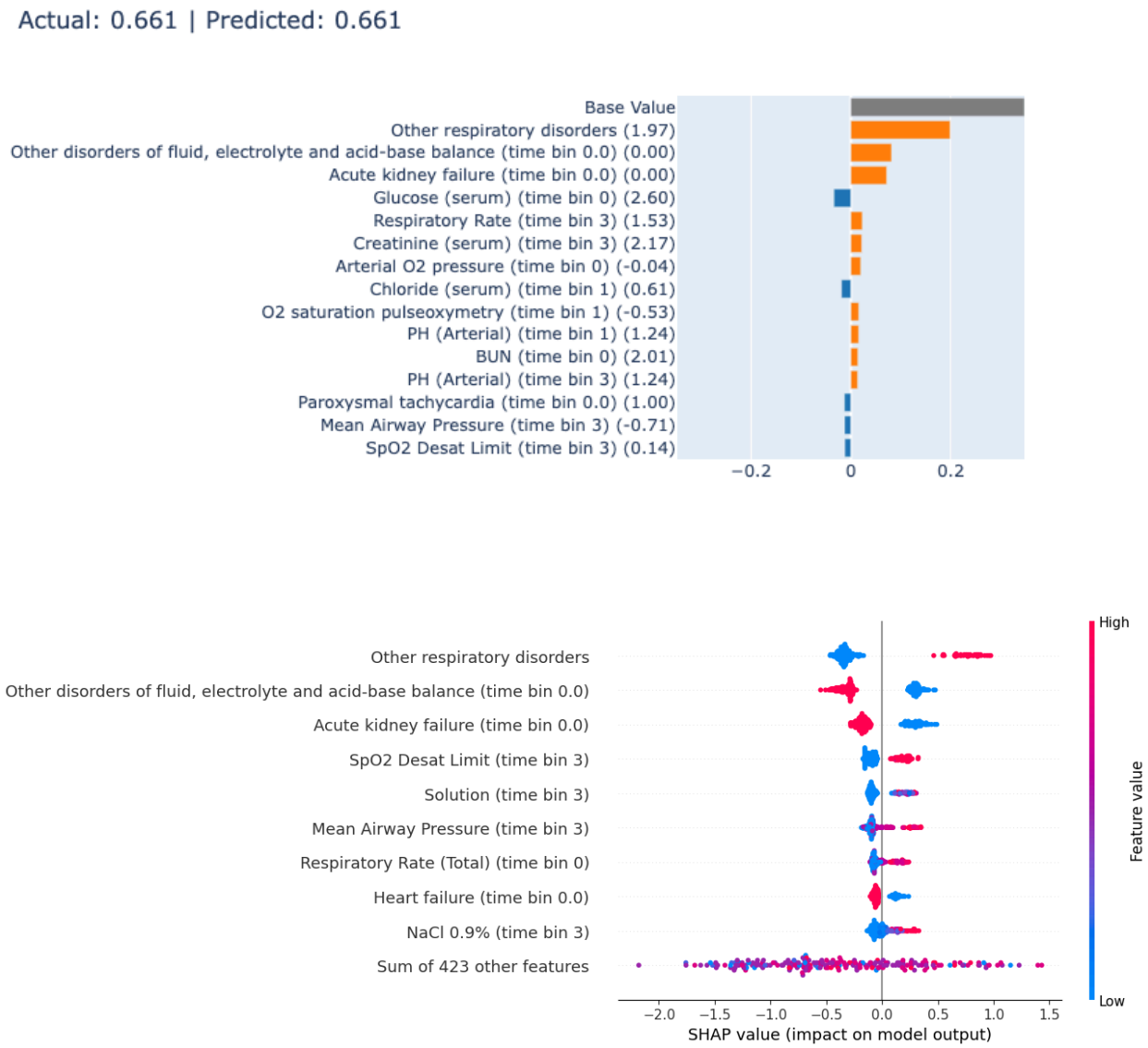


Figure 2. SHAP Beeswarm Plot of Global Feature Importance

Local and global model explainability analyses were performed to characterize the XGB model's predictions. A representative local SHAP explanation (Figure 1) demonstrated that the model's predicted risk for patient index 27 (Predicted = 0.661) was elevated by clinical factors related to respiratory and metabolic instability, including other respiratory disorders, electrolyte/acid–base abnormalities, and acute kidney failure. These features produced strong positive SHAP contributions, while lower arterial O_2 saturation, altered pH, and reduced SpO_2 desaturation limits exerted negative contributions that partially

offset the risk. Global SHAP values (Figure 2) further highlighted the overall importance of these domains, with other respiratory disorders and fluid/electrolyte imbalance emerging as the most influential predictors across the cohort. High feature values in these variables consistently pushed predictions toward higher risk, reflecting their strong association with extended ICU length of stay. Complementary ALE plots (Figure 3) revealed fairly independent relationships for several key covariates. Increasing age was associated with a small but consistent rise in predicted risk, whereas higher administration of Dextrose 5%, Propofol, and NaCl 0.9% demonstrated nonlinear effects, suggesting threshold-dependent changes in model output.

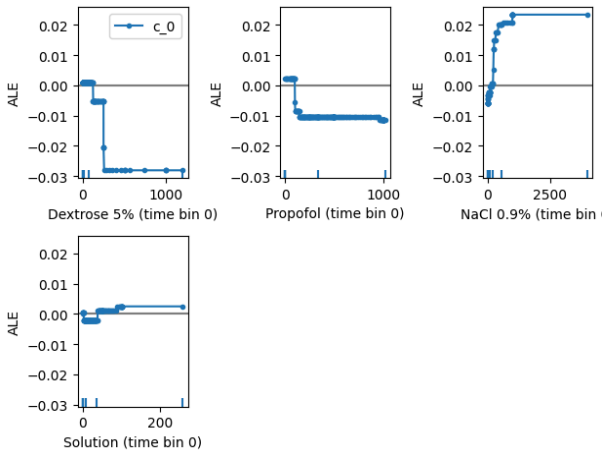


Figure 3. Accumulated Local Effects (ALE) Plots for Select Features

Fairness Evaluation

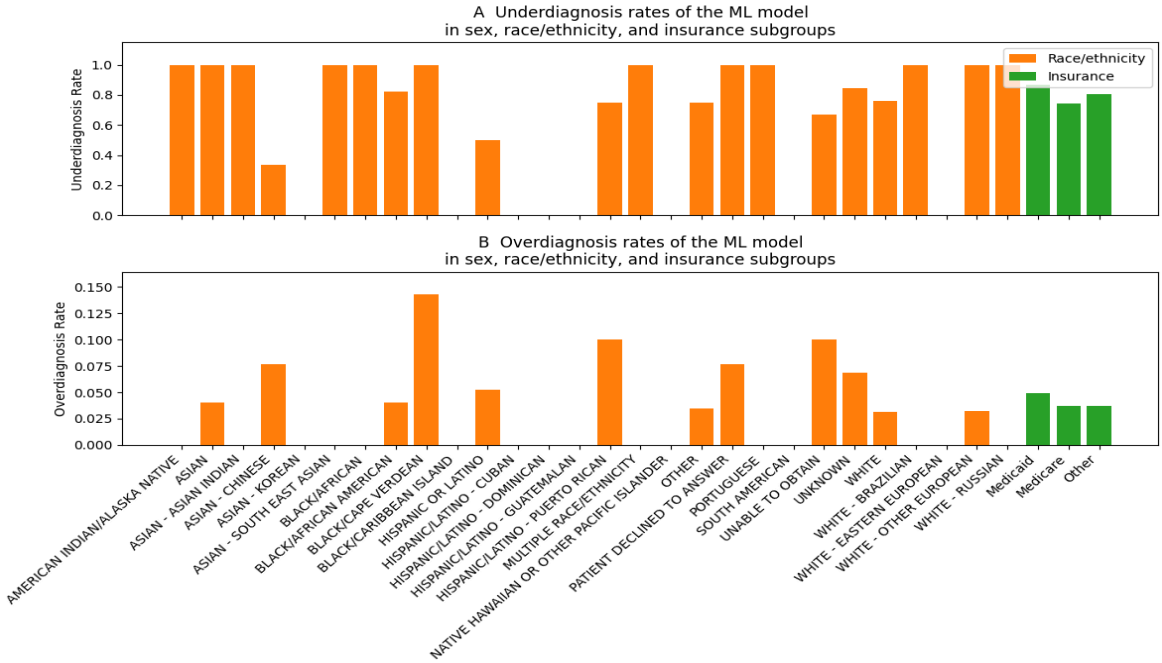


Figure 3. Evaluation of Algorithmic Bias: Underdiagnosis and Overdiagnosis Rates by Sex, Race/Ethnicity, and Insurance Subgroups

Significant performance disparities, indicative of algorithmic bias, were observed within the XGB model when evaluated across various demographic and insurance subgroups (Figure 3). Demographic parity difference revealed substantial variation across patient subgroups, particularly for ethnicity (0.286) and age groups (0.221), indicating that the model produced materially different positive prediction rates across these demographics, while gender exhibited minimal disparity (0.009). Equalized odds analysis further quantified differences in error distribution, showing large disparities for ethnicity (1.00) and age groups (0.571), with gender again showing comparatively small deviation (0.039).Underdiagnosis rates were notably elevated across most racial/ethnic groups, with several subgroups including Asian, Black/African American, and Hispanic/Latino subpopulations, showing rates approaching or reaching 1.0, indicating that the model frequently failed to identify patients who ultimately experienced extended ICU length of stay. Insurance-based disparities were also observed, though less pronounced; Medicare, Medicaid, and Other groups demonstrated moderately high underdiagnosis rates ranging from approximately 0.70 to 0.90. Overdiagnosis rates were comparatively lower across all subgroups, generally remaining below 0.15. Nonetheless, certain categories, such as Hispanic/Latino (Cuban and Puerto Rican), Asian (Chinese), and White (Brazilian) showed elevated overdiagnosis, suggesting disproportionate tendencies to incorrectly classify individuals in these populations as high risk

Distribution Shift Analysis

To assess temporal stability of the training data, we computed distributional shifts between the earliest and latest 10% of the cohort. Overall covariate drift was minimal, as indicated by a low Maximum Mean Discrepancy (MMD = 0.00335), suggesting that the joint feature distribution remained largely stable over time. Label shift was also modest: the prevalence of extended ICU LOS increased only slightly from 10.93% in the early subset to 11.28% in the late subset. Examination of demographic subpopulations revealed more notable changes. Ethnicity distribution demonstrated a measurable decline in the proportion of patients categorized as “UNKNOWN,” decreasing by approximately 5 percentage

points, indicating improved documentation or changing patient composition over time. Insurance types exhibited smaller shifts, though Medicare prevalence increased by 1.76 percentage points, while Medicaid decreased by 2.00 percentage points.

Discussion

Interpretation

This study demonstrates that early physiologic, laboratory, and diagnostic information collected within the first 12 hours of ICU admission can meaningfully predict extended length of stay. Among the models evaluated, XGBoost consistently outperformed linear approaches, reflecting its ability to capture nonlinear interactions across vital signs, metabolic markers, and comorbidity patterns. The improvement in discrimination from Project 2 to the Final Project, most notably the increase in AUPRC to 0.4329 suggests that refined preprocessing, hyperparameter tuning, and imbalance-aware optimization substantially enhanced model effectiveness. Explainability analyses corroborate the clinical plausibility of the model's predictions. Both local and global SHAP findings indicated that respiratory dysfunction, electrolyte and acid–base disturbances, and renal impairment were central drivers of prolonged ICU stays. These physiologic domains align with established clinical literature linking organ instability and metabolic derangements to delayed recovery and extended resource utilization. The model therefore appears to leverage clinically meaningful signals rather than spurious correlations.Fairness analyses revealed substantial disparities in model performance across demographic groups. The elevated underdiagnosis rates observed among several racial and ethnic minorities indicate that the model may be less sensitive to risk signals in these populations. These biases could amplify existing inequities by delaying recognition of patients in need of intensive support. The presence of disproportionate errors despite using the same physiological inputs across groups suggests that structural differences in baseline health status, documentation quality, or measurement frequency may influence model behavior.

Clinical and Informatics Relevance

Predictive models should be used to support operational planning and clinical decision-making. For such models to be adopted in high-stakes environments like the ICU, transparency is essential.

SHAP-based explanations allowed clinicians to interrogate individual predictions and verify whether they aligned with expected patterns of severity. This interpretability supports ethical deployment standards including fairness, accountability, and explainability and may reduce the mistrust commonly associated with "black-box" algorithms. Operationally, early identification of patients likely to exceed 7 days of ICU stay could enable proactive mobilization of interdisciplinary resources. Such predictions could inform staffing, equipment allocation, discharge planning, and family communication, thereby contributing to more efficient and patient-centered ICU workflows.

Limitations and Future Work

This analysis used a 50% subset of MIMIC-IV to reduce computational cost; larger samples would enhance stability. Aggregating data into 3-hour windows may mask important fluctuations. Moreover, while internal validation was strong, external validation on other EHR systems is required to ensure generalizability. Future work should apply sequence-based deep-learning models such as long short-term memory networks or Transformers to preserve full temporal resolution. Prospective evaluation in a live ICU environment could assess workflow integration, alert fatigue, and ethical oversight.

III. Methods

Data

The analysis was based on 39,329 ICU admission records corresponding to 28,022 unique patients (50% of the MIMIC-IV v2.0 database, 2008–2019), filtered to include only patients aged ≥18 years and exclude in-hospital deaths. The feature set comprised 382 variables covering clinical domains (diagnosis, procedures, events, medications) and demographics. Time-series data from the first 12 hours of admission was engineered into four 3-hour segments. Preprocessing involved standardizing diagnoses, capping outliers at the 2nd and 98th percentiles, and utilizing a combined imputation strategy (forward-fill followed by median imputation). Variables used in >80% of encounters were removed. Diagnoses, chart events, procedures, and medications were used as features. Output events were deliberately excluded from the feature set as they were deemed irrelevant for the prediction task.

Preprocessing and Feature Engineering

Feature construction was based on time-series data collected within the first 12 hours of the ICU admission, segmented into four 3-hour time buckets. Diagnoses were standardized via ICD-9 to ICD-10 conversion and ICD-10 grouping. To manage extreme data points, outliers were capped using an imputer outlier removal technique, setting the right outer threshold at the 98th percentile and the left outer threshold at the 2nd percentile. Features were processed using a combined imputation strategy: first, values were forward-filled within the patient records, followed by median imputation for any remaining missing lab or chart values across the entire dataset. After concatenating the time-series features, a final selection step removed variables that were utilized in more than 80% of the patient encounter. For the final project, the methodological pipeline was refined. Instead of a fixed split, patient-level partitions were generated through a k-fold procedure in which one fold (approximately 20%) served as the test set, and the remaining folds were divided into training and validation subsets using an internal 85/15 split, yielding effective proportions of approximately 68% training, 12% validation, and 20% testing. A structured preprocessing workflow was implemented using a ColumnTransformer: numeric variables were standardized, categorical variables were one-hot encoded, and the fit_transform operation was applied exclusively to the training set. The validation and test sets were transformed using the learned parameters to avoid data leakage.

Improved Models and Hyper-parameter Tuning

Three ML models for comparative analysis (LR, XGB Classifier, and a Linear SVM) and improved the performance. Hyperparameter tuning was performed only on the training subset and incorporated more robust search strategies: Logistic Regression and Linear SVM were tuned using 5-fold GridSearchCV, while XGB had a broader hyperparameter space explored via 40-iteration RandomizedSearchCV with 5-fold CV. The primary tuning metric was changed from ROC-AUC to AUPRC to better address the class imbalance inherent in extended length-of-stay prediction. Handling of imbalance was also improved by searching across multiple scale_pos_weight values for XGB rather than using a fixed ratio.

Explainability AI methods

Our XAI module combines model-specific and model-agnostic techniques to provide complementary global and local interpretations. For tree-based models, SHAP's TreeExplainer was used to compute feature attributions in the post-preprocessing feature space, visualized through local waterfall plots and global beeswarm plots. For linear and non-tree models, we applied Kernel SHAP on the raw feature space using model probabilities or decision functions. To provide alternative model-agnostic views, we integrated the Interpret library's Kernel SHAP and LimeTabularexplainers, generating local explanations on the fully transformed numeric data. Global feature–outcome relationships were characterized using PDP and ALE from the Alibi library to capture non-linear and interaction effects. We additionally incorporated DiCE counterfactual explanations to highlight changes associated with lower predicted risk. All outputs used a curated feature dictionary to map engineered feature names to clinically meaningful labels.

Fairness evaluation

Model fairness was assessed across key demographic groups (ethnicity, gender, and decade-based age bins) using demographic parity and equalized odds. Binary predictions were generated using a 0.5 probability threshold. Demographic parity was evaluated by comparing subgroup-specific positive prediction rates, indicating whether the model assigns elevated risk disproportionately across groups. Equalized odds was assessed through subgroup-level TPRand FPR, revealing disparities in correct and incorrect identification of extended ICU stays. All fairness metrics were computed using Fairlearn's MetricFrame.

Distribution shift

To assess temporal robustness and identify potential distribution shift, we compared the earliest and latest 10% of ICU admissions in the cohort. Admission timestamps were standardized and chronologically sorted to define early and late-period subsamples. Global distributional drift was quantified using Maximum Mean Discrepancy (MMD) with an RBF kernel, providing a nonparametric measure of divergence in the joint feature space. We evaluated label shift by comparing the prevalence of extended ICU length of stay between periods. Feature-level covariate drift was examined using

two-sample Kolmogorov–Smirnov (KS) tests applied to each numeric predictor, with variables ranked by KS statistic to highlight the most temporally unstable features. Finally, demographic shift was assessed by comparing normalized category frequencies for ethnicity, gender, and insurance type across the two time periods.

V. Conclusion

Machine learning methods can accurately predict extended ICU length of stay using only the first 12 hours of data. Among the models tested, XGB achieved the best trade-off between discrimination and interpretability (AUPRC= 0.4329). By embedding these interpretable models into ICU dashboards, hospitals could anticipate patient trajectories, optimize staffing, and reduce bottlenecks in critical care delivery. The project underscores the potential of transparent AI to enhance both operational efficiency and patient outcomes in modern healthcare settings.

AI Use Disclosure

The coding portion & minimal written composition of this project was used as assistance from ChatGPT and Google Gemini (refer to Supplemental).

References

Google. (2025). *Gemini (2.0 Pro)* [Large language model]. https://gemini.google.com/

Johnson, A. E. W., Bulgarelli, L., Pollard, T. J., Horng, S., Celi, L. A., & Mark, R. G. (2023). *MIMIC-IV (version 2.2)* [Data set]. PhysioNet. https://doi.org/10.13026/6mm1-mj77

Johnson, A., Bulgarelli, L., Pollard, T., Horng, S., Celi, L. A., & Mark, R. (2022). *MIMIC-IV (version 2.0)* [Dataset]. PhysioNet. https://doi.org/10.13026/7vcr-e114

OpenAI. (2025). *ChatGPT (GPT-5)* [Large language model]. OpenAI. https://chat.openai.com

Wachter, S., Mittelstadt, B., & Russell, C. (2018). *Counterfactual explanations without opening the black box: Automated decisions and the GDPR. Harvard Journal of Law & Technology, 31*(2), 841–887.

Supplemental

Column 1	Training Set	Validation Set	Test Set
Demographics			
Age (Mean (SD))	63.20 (16.98)	62.99 (16.99)	63.09 (16.91)
Male	13560 (56.2%)	3389 (56.2%)	4204 (55.8%)
American Indian or Alaska Native	48 (0.2%)	21 (0.3%)	20 (0.3%)
Asian	666 (2.8%)	176 (2.9%)	227 (3.0%)
Black or African American	2412 (10.0%)	587 (9.7%)	754 (10.0%)
Hispanic or Latino	875 (3.6%)	236 (3.9%)	257 (3.4%)
Other/Multiple/Unknown	3581 (14.8%)	927 (15.4%)	1119 (14.8%)
Pacific Islander	32 (0.1%)	11 (0.2%)	12 (0.2%)
White	16506 (68.4%)	4072 (67.5%)	5149 (68.3%)
Insurance			
Medicaid	1874 (7.8%)	489 (8.1%)	579 (7.7%)
Medicare	10734 (44.5%)	2673 (44.3%)	3349 (44.4%)
Other	11512 (47.7%)	2868 (47.6%)	3610 (47.9%)
Comorbidities			
Other anemias	6176 (25.6%)	1531 (25.4%)	1873 (24.8%)
Type 2 diabetes mellitus	6742 (28.0%)	1709 (28.3%)	2147 (28.5%)
Disorders of lipoprotein metabolism and other lipidemias	9869 (40.9%)	2559 (42.4%)	3108 (41.2%)
Other disorders of fluid, electrolyte and acid-base balance	9536 (39.5%)	2449 (40.6%)	3006 (39.9%)
Essential (primary) hypertension	9964 (41.3%)	2533 (42.0%)	3094 (41.0%)
Chronic ischemic heart disease	7384 (30.6%)	1857 (30.8%)	2282 (30.3%)
Paroxysmal tachycardia	5870 (24.3%)	1524 (25.3%)	1899 (25.2%)
Heart failure	6378 (26.4%)	1599 (26.5%)	2029 (26.9%)
Other respiratory disorders	4919 (20.4%)	1214 (20.1%)	1516 (20.1%)
Acute kidney failure	7114 (29.5%)	1753 (29.1%)	2278 (30.2%)
Chronic kidney disease (CKD)	4989 (20.7%)	1207 (20.0%)	1576 (20.9%)
Encounter for other aftercare and medical care	5604 (23.2%)	1410 (23.4%)	1748 (23.2%)

Table S1. Summary Statistics

Figure S2. Beewarm

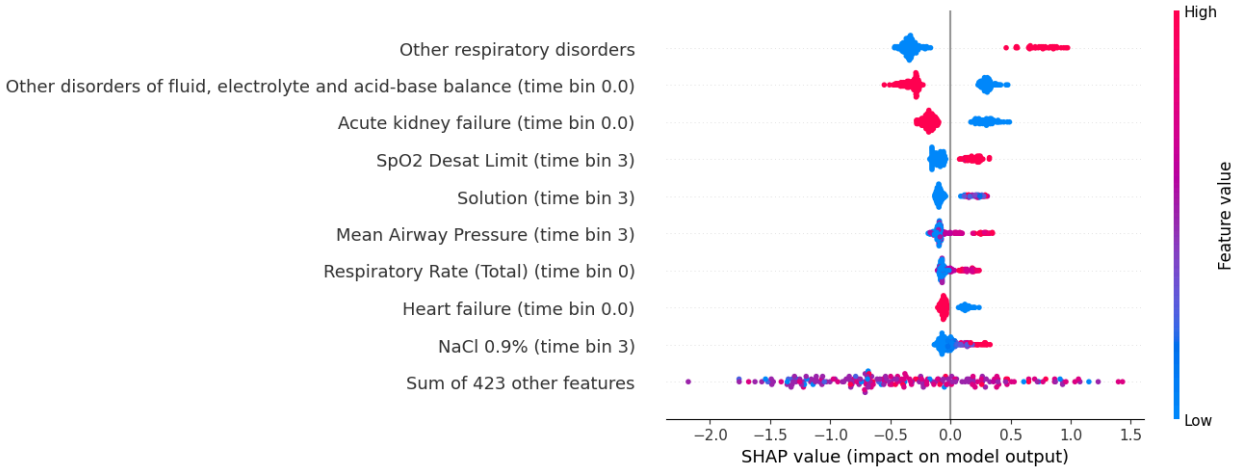


Figure S2. LIME

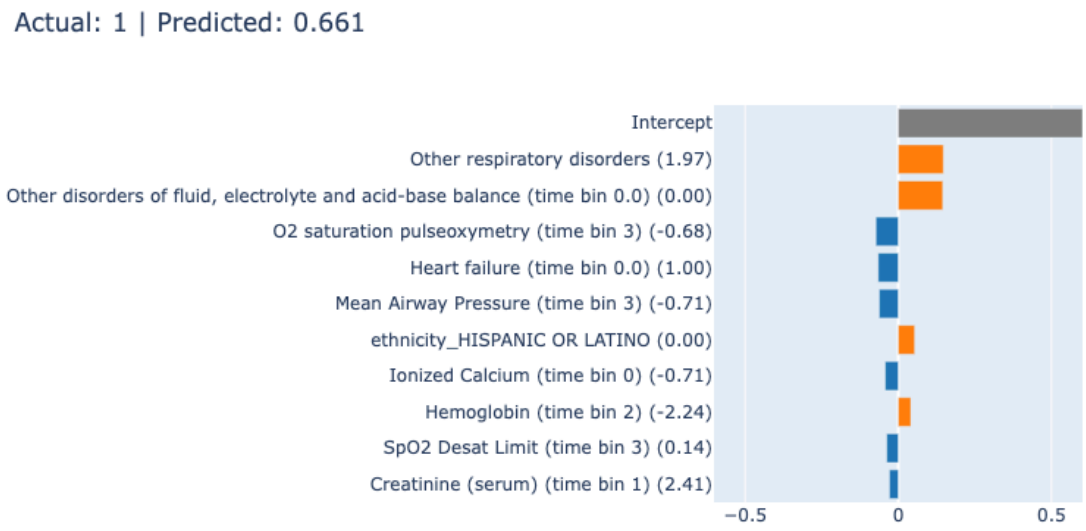
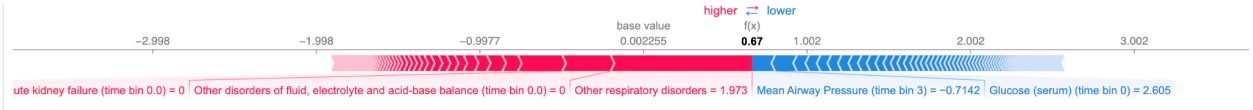


Figure S3. Force plot



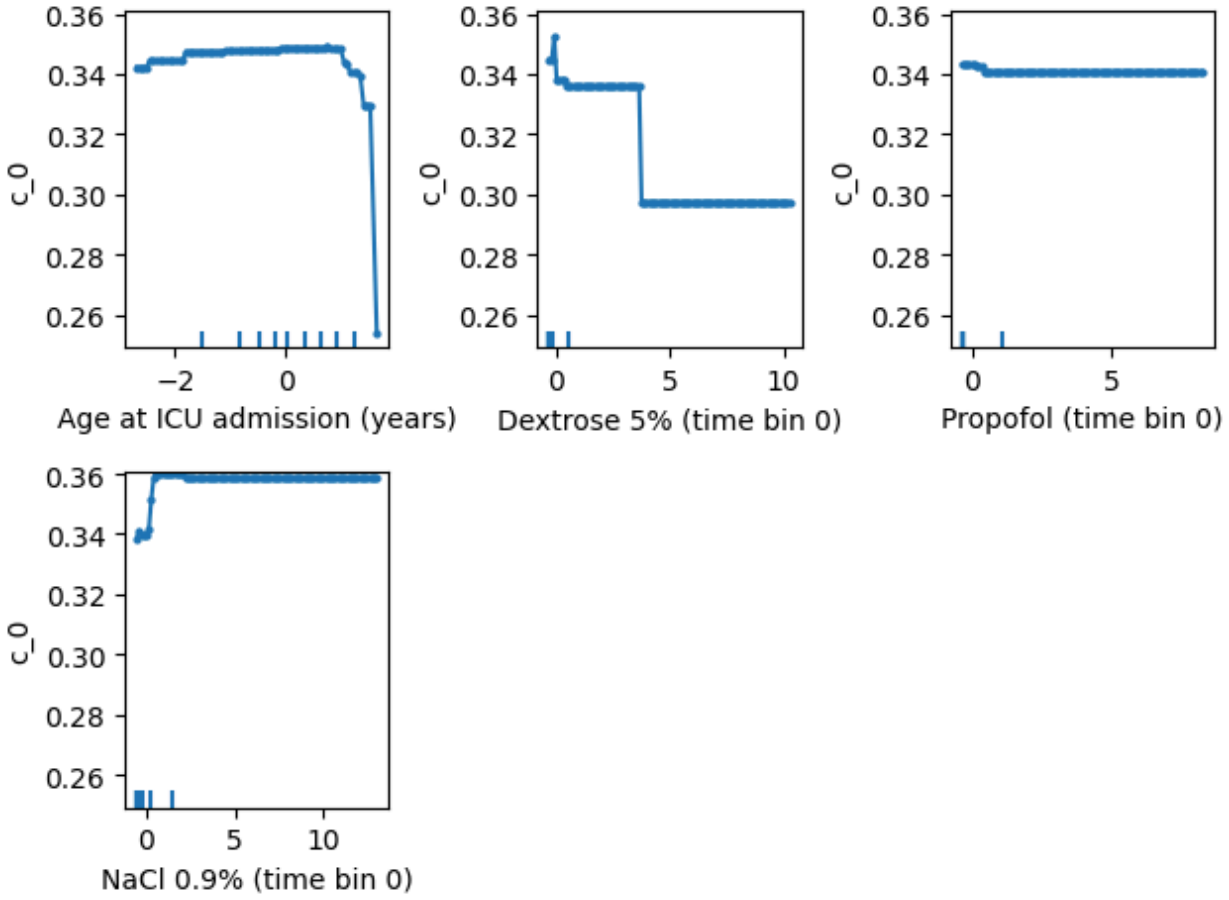


Figure S3. PDP

Google Gemini prompt: Create a coding template machine learning pipeline for XGBoost and Support Vector Machine

How the answer was adopted: Imputation for missing values was not used as it was already been used in the machine learning pipeline. The template for the SVM and XGBoost was used to help set up the code.

Answer:

```
#
==============================================================================
==
# 1. IMPORTS
#
==============================================================================
==
import pandas as pd
import numpy as np

# Preprocessing & Pipeline
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer

# Models
from sklearn.svm import SVC             # TODO: For Classification
# from sklearn.svm import SVR           # TODO: For Regression
from xgboost import XGBClassifier       # TODO: For Classification
```

```
# from xgboost import XGBRegressor       # TODO: LOR Regression


# Evaluation
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix # TODO: For
Classification
# from sklearn.metrics import mean_squared_error, r2_score                    # TODO: For Regression


# Model Saving
import joblib
import warnings


warnings.filterwarnings('ignore')


#
==============================================================================
==
# 2. CONFIGURATION
#
==============================================================================
==
# Set a random state for reproducibility
RANDOM_STATE = 42


# Define file paths and column names
# TODO: Update these variables
DATA_FILE = 'your_data.csv'
```

```
TARGET_COLUMN = 'your_target_variable'
MODEL_SVM_PATH = 'svm_pipeline.joblib'
MODEL_XGB_PATH = 'xgb_pipeline.joblib'



#
==============================================================================
==
# 3. LOAD DATA
#
==============================================================================
==
print("Loading data...")
try:
    data = pd.read_csv(DATA_FILE)
except FileNotFoundError:
    print(f"Error: Data file '{DATA_FILE}' not found.")
    # In a real script, you might exit or raise an error here
    # For this template, we'll create dummy data
    data = pd.DataFrame({
        'numeric_feat_1': np.random.rand(100),
        'numeric_feat_2': np.random.rand(100),
        'categorical_feat_1': np.random.choice(['A', 'B', 'C'], 100),
        'categorical_feat_2': np.random.choice(['X', 'Y'], 100),
        'your_target_variable': np.random.randint(0, 2, 100)
    })
```

```
    print("Using dummy data for demonstration.")


#
==========================================================================
==
# 4. DEFINE FEATURES (X) AND TARGET (y)
#
==========================================================================
==
print("Defining features and target...")
X = data.drop(TARGET_COLUMN, axis=1)
y = data[TARGET_COLUMN]


# TODO: Identify your numeric and categorical features
# We will try to infer them, but manual definition is safer.
numeric_features = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_features = X.select_dtypes(include=['object', 'category']).columns.tolist()


print(f"Numeric features: {numeric_features}")
print(f"Categorical features: {categorical_features}")



#
==========================================================================
==
# 5. TRAIN-TEST SPLIT
```

```
#
==========================================================================
==
print("Splitting data into train and test sets...")
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y if y.nunique() > 1 else None
)


#
==========================================================================
==
# 6. PREPROCESSING PIPELINE
#
==========================================================================
==
print("Building preprocessing pipeline...")


# Create a pipeline for numeric features
# This will impute missing values (with the median) and then scale the data
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])


# Create a pipeline for categorical features
# This will impute missing values (with the most frequent value) and then one-hot encode
```

```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Combine preprocessing steps using ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ],
    remainder='passthrough' # Keep any columns not specified
)



#
==============================================================================
==
# 7. DEFINE MODEL PIPELINES
#
==============================================================================
==
print("Defining full model pipelines...")

# --- Pipeline 1: Support Vector Machine (SVM) ---
```

```
# TODO: Choose SVC for Classification or SVR for Regression
model_svm = SVC(probability=True, random_state=RANDOM_STATE) # Classification
# model_svm = SVR()                                  # Regression

# Chain the preprocessor and the model together
pipeline_svm = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', model_svm)
])

# --- Pipeline 2: XGBoost ---

# TODO: Choose XGBClassifier or XGBRegressor
model_xgb = XGBClassifier(
    random_state=RANDOM_STATE,
    use_label_encoder=False,  # Recommended setting
    eval_metric='logloss'     # For classification. Use 'rmse' for regression.
)
# model_xgb = XGBRegressor(random_state=RANDOM_STATE, eval_metric='rmse')

# Chain the preprocessor and the model together
pipeline_xgb = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', model_xgb)
])
```

```
#
==========================================================================
==
# 8. HYPERPARAMETER TUNING (OPTIONAL BUT RECOMMENDED)
#
==========================================================================
==
print("Setting up hyperparameter grids...")


# --- SVM Parameter Grid ---
# Note: Parameters are prefixed with 'model__' to tell the pipeline
# which step to apply the parameter to.
param_grid_svm = {
    'model__C': [0.1, 1, 10],
    'model__kernel': ['linear', 'rbf'],
    'model__gamma': ['scale', 'auto'] # Ignored by 'linear' kernel
}


# --- XGBoost Parameter Grid ---
param_grid_xgb = {
    'model__n_estimators': [100, 200, 300],
    'model__learning_rate': [0.01, 0.1, 0.2],
    'model__max_depth': [3, 5, 7]
}
```

```
# --- Initialize GridSearchCV ---
# We use GridSearchCV to find the best parameters based on cross-validation.
# 'cv=3' means 3-fold cross-validation. Increase for more robust, but slower, tuning.
# 'n_jobs=-1' uses all available CPU cores.
# 'scoring' can be 'accuracy', 'f1', 'roc_auc' (Classification) or 'r2', 'neg_mean_squared_error' (Regression)
# TODO: Choose appropriate scoring
scoring_metric = 'accuracy' # For Classification
# scoring_metric = 'neg_mean_squared_error' # For Regression

grid_search_svm = GridSearchCV(
    pipeline_svm, param_grid_svm, cv=3, scoring=scoring_metric, n_jobs=-1, verbose=1
)

grid_search_xgb = GridSearchCV(
    pipeline_xgb, param_grid_xgb, cv=3, scoring=scoring_metric, n_jobs=-1, verbose=1
)



#
==========================================================================
==
# 9. TRAIN AND EVALUATE MODELS
#
==========================================================================
==
```

```
# --- Train and Evaluate SVM ---
print("\n--- Training SVM ---")
grid_search_svm.fit(X_train, y_train)

print(f"Best SVM parameters: {grid_search_svm.best_params_}")
print(f"Best SVM cross-validation score ({scoring_metric}): {grid_search_svm.best_score_:.4f}")

# Get the best model
best_svm = grid_search_svm.best_estimator_

# Evaluate on the test set
print("\nEvaluating SVM on Test Set:")
y_pred_svm = best_svm.predict(X_test)

# TODO: Use appropriate evaluation metrics
# --- Classification Metrics ---
print(f"Accuracy: {accuracy_score(y_test, y_pred_svm):.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred_svm))

# --- Regression Metrics ---
# print(f"R-squared: {r2_score(y_test, y_pred_svm):.4f}")
# print(f"Mean Squared Error (MSE): {mean_squared_error(y_test, y_pred_svm):.4f}")


# --- Train and Evaluate XGBoost ---
```

```
print("\n--- Training XGBoost ---")
grid_search_xgb.fit(X_train, y_train)

print(f"Best XGBoost parameters: {grid_search_xgb.best_params_}")
print(f"Best XGBoost cross-validation score ({scoring_metric}): {grid_search_xgb.best_score_:.4f}")

# Get the best model
best_xgb = grid_search_xgb.best_estimator_

# Evaluate on the test set
print("\nEvaluating XGBoost on Test Set:")
y_pred_xgb = best_xgb.predict(X_test)

# TODO: Use appropriate evaluation metrics
# --- Classification Metrics ---
print(f"Accuracy: {accuracy_score(y_test, y_pred_xgb):.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred_xgb))

# --- Regression Metrics ---
# print(f"R-squared: {r2_score(y_test, y_pred_xgb):.4f}")
# print(f"Mean Squared Error (MSE): {mean_squared_error(y_test, y_pred_xgb):.4f}")
```

```
#
==========================================================================
==
# 10. SAVE FINAL MODELS
#
==========================================================================
==
print("\nSaving final model pipelines...")
joblib.dump(best_svm, MODEL_SVM_PATH)
joblib.dump(best_xgb, MODEL_XGB_PATH)

print(f"SVM pipeline saved to {MODEL_SVM_PATH}")
print(f"XGBoost pipeline saved to {MODEL_XGB_PATH}")
print("Script finished.")

#
==========================================================================
==
# 11. (EXAMPLE) LOAD AND USE A SAVED MODEL
#
==========================================================================
==
# print("\nLoading and using saved model for a new prediction...")
#
# # Load the model
# loaded_model = joblib.load(MODEL_XGB_PATH)
```

```
#
# # Create new, unseen data (must have the same column names as training data)
# # TODO: Replace with your actual new data
# new_data = pd.DataFrame({
#     'numeric_feat_1': [0.5],
#     'numeric_feat_2': [0.2],
#     'categorical_feat_1': ['B'],
#     'categorical_feat_2': ['Y']
# })
#
# # Make predictions
# # The loaded pipeline will automatically apply all preprocessing steps
# prediction = loaded_model.predict(new_data)
# prediction_proba = loaded_model.predict_proba(new_data) # For classification
#
# print(f"Prediction for new data: {prediction[0]}")
# print(f"Prediction probabilities: {prediction_proba[0]}") # For classification
```